

SHA, MD5

```

import java.util.Scanner;
import java.security.MessageDigest;
public class Hash{
    public static void main(String[] args) throws Exception{
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter a message: ");
        String msg = sc.nextLine();
        MessageDigest md = MessageDigest.getInstance("MD5");
        byte[] md5 = md.digest(msg.getBytes());
        System.out.print("\nMD5:");
        for(byte b : md5)
            System.out.printf("%02x", b);

        MessageDigest sha = MessageDigest.getInstance("SHA-256");
        byte[] sh = sha.digest(msg.getBytes());
        System.out.print("\nSHA-256:");
        for(byte b : sh)
            System.out.printf("%02x", b);
    }
}

```

ELGAMAL

```

#include<iostream>
using namespace std;
long long power(long long a,long long b,long long mod)
{
    long long res = 1;
    while(b)
    {
        if(b%2) res = (res*a)% mod;
        a = (a*a)%mod;
        b/=2;
    }
    return res;
}
int main()
{
    long long p,g,x,h,k,m,c1,c2,s,m_dec;
    p=43, g=6, x=5, k=7;
    h = power(g,x,p);
    cout<<"Enter a message:";
    cin>>m;
    c1 = power(g,k,p);
    c2 = (m*power(h,k,p))%p;
    s = power(c1,x,p);
    m_dec = (c2 * power(s,p-2,p))%p;
    cout<<"Cipher:"<<c1<<" "<<c2;
    cout<<"Decrypted msg:"<<m_dec; }

```

DIFFIE-HELLMAN

```

#include<iostream>
using namespace std;
long long power(long long a,long long b,long long mod)
{
    long long res = 1;
    while(b)
    {
        if(b%2) res = (res*a)% mod;
        a = (a*a)%mod;
        b/=2;
    }
    return res;
}
int main()
{
    long long p,g,a,b,A,B,keyA,keyB;
    p=23,g=5,a=6,b=15;
    A=power(g,a,p);
    B=power(g,b,p);
    keyA=power(B,a,p);
    keyB=power(A,b,p);
    cout<<A<<" "<<B<<" "<<keyA<<" "<<keyB;
}

```

RSA

```

#include<iostream>
using namespace std;
long long power(long long a,long long b,long long mod)
{
    long long res = 1;
    while(b)
    {
        if(b%2) res = (res*a)% mod;
        a = (a*a)%mod;
        b/=2;
    }
    return res;
}
int main()
{
    long long p,q,n,phi,e,d,m,c,m_dec;
    p=11,q=3,d=7;
    n=p*q;
    phi=(p-1)*(q-1);
    cout<<"Enter message:";
    cin>>m;
    c = power(m,e,n);
    m_dec = power(c,d,n);
    cout<<c<<" "<<m_dec; }

```

DES

```

import java.util.Scanner;
import javax.crypto.Cipher;
import javax.crypto.spec.SecretKeySpec;
public class DES {
    public static void main(String[] args) throws Exception {
        Scanner sc = new Scanner(System.in);
        String msg;
        System.out.print("Enter a message:");
        msg = sc.nextLine();
        String key = "12345678";
        SecretKeySpec ks = new SecretKeySpec(key.getBytes(),"DES");
        Cipher cipher = Cipher.getInstance("DES");

        cipher.init(Cipher.ENCRYPT_MODE,ks);
        byte[] enc = cipher.doFinal(msg.getBytes());
        System.out.print("Encrypted message: " + new String(enc));

        cipher.init(Cipher.DECRYPT_MODE,ks);
        byte[] dec = cipher.doFinal(enc);
        System.out.print("Decrypted message: " + new String(dec));
    }
}

```

ROW AND COLUMN TRANSFORMATION

```

#include<iostream>
using namespace std;
int main()
{
    int r = 3, c= 4;
    int mat[10][10];
    string txt = "";
    cout<<"Enter a message:";
    cin>>txt;
    int k=0;
    int i,j;
    for(i=0;i<r;i++)
    {
        for(j=0;j<c;j++)
        {
            mat[i][j] = (k<txt.length()) ? txt[k++] : 'X';
        }
    }
    string enc = "";
    for(j=0;j<c;j++)
    {
        for(i=0;i<r;i++)
        {

```

```

        enc += mat[i][j];
    }
}
k=0;
for(j=0;j<c;j++)
{
    for(i=0;i<r;i++)
    {
        mat[i][j] = enc[k++];
    }
}
string dec = "";
for(i=0;i<r;i++)
{
    for(j=0;j<c;j++)
    {
        dec += mat[i][j];
    }
}
cout<<"Encrypted message: "<<enc;
cout<<"Decrypted message: "<<dec;
}

```

RAIL FENCE TRANSFORMATION

```

#include <iostream>
using namespace std;

int main() {
    string text;
    int rails = 3;

    cout << "Enter text: ";
    cin >> text;

    string rail[10];

    int row = 0, dir = 1;

    // Encryption (zig-zag fill)
    for (char ch : text) {
        rail[row] += ch;
        if (row == 0) dir = 1;
        else if (row == rails - 1) dir = -1;
        row += dir;
    }

    string cipher = "";
    for (int i = 0; i < rails; i++)
        cipher += rail[i];
}

```

```

cout << "Encrypted: " << cipher << endl;

// Decryption
int len = text.length();
char mat[10][100] = {0};

row = 0; dir = 1;
for (int i = 0; i < len; i++) {
    mat[row][i] = '*';
    if (row == 0) dir = 1;
    else if (row == rails - 1) dir = -1;
    row += dir;
}

int k = 0;
for (int i = 0; i < rails; i++)
    for (int j = 0; j < len; j++)
        if (mat[i][j] == '*')
            mat[i][j] = cipher[k++];

string plain = "";
row = 0; dir = 1;
for (int i = 0; i < len; i++) {
    plain += mat[row][i];
    if (row == 0) dir = 1;
    else if (row == rails - 1) dir = -1;
    row += dir;
}

cout << "Decrypted: " << plain;
}

```

VIGNERE CIPHER

```

#include <iostream>
using namespace std;

int main() {
    string text, key;
    cout << "Enter text: ";
    cin >> text;
    cout << "Enter key: ";
    cin >> key;

    // Make key same length as text
    for (int i = 0; key.length() < text.length(); i++)
        key += key[i];

    string cipher = "", plain = "";

```

```

// Encryption
for (int i = 0; i < text.length(); i++)
    cipher += char((text[i] + key[i]) % 26 + 'A');

cout << "Encrypted: " << cipher << endl;

// Decryption
for (int i = 0; i < cipher.length(); i++)
    plain += char((cipher[i] - key[i] + 26) % 26 + 'A');

cout << "Decrypted: " << plain;
}

```

HILL CIPHER

```

#include <iostream>
using namespace std;

int mod26(int x){ x%=26; if(x<0) x+=26; return x; }

// Multiply 2x2 matrix with vector
void multiply(int key[2][2], int p[2], int c[2]){
    for(int i=0;i<2;i++){
        c[i]=0;
        for(int j=0;j<2;j++)
            c[i]+=key[i][j]*p[j];
        c[i]=mod26(c[i]);
    }
}

// Find inverse of key
void inverseKey(int key[2][2], int inv[2][2]){
    int det = mod26(key[0][0]*key[1][1] - key[0][1]*key[1][0]);
    int detInv;
    for(int i=1;i<26;i++)
        if(mod26(det*i)==1) detInv=i;

    inv[0][0]= mod26( key[1][1]*detInv);
    inv[0][1]= mod26(-key[0][1]*detInv);
    inv[1][0]= mod26(-key[1][0]*detInv);
    inv[1][1]= mod26( key[0][0]*detInv);
}

string process(string text, int key[2][2]){
    if(text.length()%2!=0) text+='X';
    string result="";
    for(int i=0;i<text.length();i+=2){
        int p[2]={text[i]-'A', text[i+1]-'A'};
        int c[2];
    }
}

```

```

        multiply(key,p,c);
        result+=char(c[0]+'A');
        result+=char(c[1]+'A');
    }
    return result;
}

int main(){
    int key[2][2]={ {3,3},{2,5}}; // Safe key
    int inv[2][2];

    string text;
    cout<<"Enter UPPERCASE text: ";
    cin>>text;

    string cipher = process(text,key);
    cout<<"Encrypted: "<<cipher<<endl;

    inverseKey(key,inv);
    string plain = process(cipher,inv);
    cout<<"Decrypted: "<<plain<<endl;
}

```

PLAYFAIR CIPHER

```

#include <iostream>
#include <cctype>
using namespace std;

char M[5][5] = {
    {'M','O','N','A','R'},
    {'C','H','Y','B','D'},
    {'E','F','G','I','K'},
    {'L','P','Q','S','T'},
    {'U','V','W','X','Z'}
};

void pos(char ch,int &r,int &c){
    if(ch=='J') ch='I';
    for(int i=0;i<5;i++)
        for(int j=0;j<5;j++)
            if(M[i][j]==ch){ r=i; c=j; return; }
}

string format(string s){
    string t="";
    for(char c:s) if(isalpha(c)) t+=toupper(c);
    for(int i=0;i<t.length();i+=2){
        if(i+1==t.length() || t[i]==t[i+1])
            t.insert(i+1,"X");
    }
}

```

```

    }
    if(t.length()%2) t+='X';
    return t;
}

string encrypt(string s){
    s=format(s);
    string out="";
    for(int i=0;i<s.length();i+=2){
        int r1,c1,r2,c2;
        pos(s[i],r1,c1);
        pos(s[i+1],r2,c2);

        if(r1==r2){
            out+=M[r1][(c1+1)%5];
            out+=M[r2][(c2+1)%5];
        }
        else if(c1==c2){
            out+=M[(r1+1)%5][c1];
            out+=M[(r2+1)%5][c2];
        }
        else{
            out+=M[r1][c2];
            out+=M[r2][c1];
        }
    }
    return out;
}

string decrypt(string s){
    string out="";
    for(int i=0;i<s.length();i+=2){
        int r1,c1,r2,c2;
        pos(s[i],r1,c1);
        pos(s[i+1],r2,c2);

        if(r1==r2){
            out+=M[r1][(c1+4)%5];
            out+=M[r2][(c2+4)%5];
        }
        else if(c1==c2){
            out+=M[(r1+4)%5][c1];
            out+=M[(r2+4)%5][c2];
        }
        else{
            out+=M[r1][c2];
            out+=M[r2][c1];
        }
    }
    return out;
}

```



```

}

int main(){
    int ch; string text;
    cout<<"1.Encrypt 2.Decrypt: ";
    cin>>ch; cin.ignore();

    if(ch==1){
        cout<<"Enter Plaintext: ";
        getline(cin,text);
        cout<<"Cipher: "<<encrypt(text);
    } else {
        cout<<"Enter Ciphertext: ";
        getline(cin,text);
        for(char &c:text) c=toupper(c);
        cout<<"Plain: "<<decrypt(text);
    }
}

```

CAESAR CIPHER

```

#include <iostream>
#include <cctype>
using namespace std;

string shiftText(string s, int k){
    string r="";
    for(char c:s){
        if(isalpha(c)){
            char base = isupper(c) ? 'A' : 'a';
            r += char((c - base + k + 26) % 26 + base);
        } else r += c;
    }
    return r;
}

int main(){
    string text; int k;
    cout<<"Enter text: ";
    getline(cin,text);
    cout<<"Enter shift: ";
    cin>>k;

    string enc = shiftText(text, k);
    cout<<"Encrypted: "<<enc<<endl;
    cout<<"Decrypted: "<<shiftText(enc, -k)<<endl;
}

```